



Java Backend Developer

Complete Roadmap 2025-2027

Zero to Production-Ready Backend Engineer

12

Phases

100+

Days

15+

Projects

2027

Target

Java

Spring Boot

Microservices

Docker

AWS

Kafka

K8s

AI

Personalized for Arshad | Target: Java Backend Role @ Fintech | 8-12 LPA

Table of Contents

Phase 0	Foundation & Environment Setup	Beginner
Phase 1	Core Java — OOP & Collections	Beginner
Phase 2	Spring Boot & REST APIs	Beginner-Intermediate
Phase 3	SQL, JDBC, JPA & Hibernate	Intermediate
[MAJOR PROJECT 1]	FinTrack — Finance Tracker API	---
Phase 4	Dev Tools, Git, Design Patterns	Intermediate
Phase 5	Low-Level Design (LLD)	Intermediate
Phase 6	Security, JWT, OAuth2	Intermediate-Advanced
Phase 7	High-Level Design (HLD)	Intermediate-Advanced
[MAJOR PROJECT 2]	JobBoard — Job Portal Backend	---
Phase 8	DSA for Interviews (Parallel Track)	Ongoing
Phase 9	Docker & DevOps / CI-CD	Intermediate
Phase 10	AWS Cloud Deployment	Intermediate
Phase 11	Microservices & Kafka & K8s	Advanced
Phase 12	AI Integration with Spring AI	Advanced
[MAJOR PROJECT 3]	PayFlow — Fintech Microservices	---
[TIMELINE]	Complete 2025-2027 Roadmap Overview	---

Environment Setup

- | | |
|----------------------------------|----------------------------|
| ✓ Install JDK 21 (LTS) | ✓ Postman (API testing) |
| ✓ IntelliJ IDEA Community/Ultime | ✓ VS Code (optional) |
| ✓ Git + GitHub account | ✓ Maven installed globally |

Java Basics — Syntax & Mindset

- | | |
|---------------------------------------|---|
| ✓ How JVM works (compile to bytecode) | ✓ Variables & Data Types (int, String, boolean) |
| ✓ Platform independence concept | ✓ Type casting & wrapper classes |
| ✓ Hello World, main() method | ✓ Input/Output (Scanner class) |

Control Flow

- | | |
|------------------------------|--------------------|
| ✓ if / else / else if | ✓ break & continue |
| ✓ switch statement | ✓ Ternary operator |
| ✓ for, while, do-while loops | ✓ Nested loops |

MINI PROJECT: Simple Calculator CLI

Build a command-line calculator that handles +, -, *, / with input validation.

Tech: Core Java (no frameworks)

Resources → YouTube:

- Telusko Java Full Course (Hindi)
- Kunal Kushwaha Java Basics

Resources → Practice:

- HackerRank Java track (Easy problems)

Object-Oriented Programming (OOP)

- | | |
|---|---|
| ✓ Classes & Objects | ✓ Abstraction (abstract classes) |
| ✓ Constructors (default, parameterized) | ✓ Interfaces & default methods |
| ✓ this keyword | ✓ Polymorphism (runtime & compile-time) |
| ✓ Inheritance & super keyword | ✓ static & final keywords |
| ✓ Method Overriding & Overloading | ✓ Access modifiers (public, private, protected) |
| ✓ Encapsulation (getters/setters) | |

Collections Framework

- | | |
|---------------------------|----------------------------|
| ✓ ArrayList & LinkedList | ✓ Iterator & for-each loop |
| ✓ HashMap & LinkedHashMap | ✓ Comparable & Comparator |

✓ HashSet & TreeSet	✓ Collections utility class
✓ Stack & Queue	✓ Generics (List, Map)

Exception Handling

✓ try / catch / finally	✓ Custom Exception classes
✓ Checked vs Unchecked exceptions	✓ Multi-catch blocks
✓ throw & throws keywords	✓ Exception chaining

Java 8+ Modern Features

✓ Lambda expressions	✓ Method references (::)
✓ Functional Interfaces (Predicate, Function, Consumer)	✓ Default & static interface methods
✓ Stream API (filter, map, collect, reduce)	✓ var keyword (Java 10+)
✓ Optional class	✓ Records (Java 16+)

MINI PROJECT: Student Management System

CLI app to Add/Update/Delete/Search students using ArrayList & HashMap. Sort by marks using Comparator, validate with exceptions, use Streams for filtering.

Tech: Core Java, OOP, Collections, Streams

Resources → YouTube:

- Telusko OOP Playlist
- Java Brains — Lambda & Streams

Resources → Books:

- Head First Java (free PDF online)

Resources → Practice:

- LeetCode Easy — Arrays & Strings (10 problems)

PHASE
2

Spring Framework Core & Spring Boot

Weeks 3-4 | Days 19-30 | Beginner-Intermediate

Weeks 3-4 | Days 19-30

Spring Core Concepts

✓ IoC (Inversion of Control) — what & why	✓ Bean lifecycle & scope (singleton, prototype)
✓ DI (Dependency Injection) — constructor & setter	✓ ApplicationContext & Spring Container
✓ @Component, @Service, @Repository, @Controller	✓ @Configuration & @Bean
✓ @Autowired — field, constructor, setter	✓ @Qualifier & @Primary

Spring Boot Essentials

✓ Spring Boot vs Spring Framework	✓ Embedded Tomcat server
✓ spring-boot-starter-* magic	✓ Auto-configuration concept
✓ application.properties / application.yml	✓ Maven project structure (pom.xml)
✓ @SpringBootApplication annotation	✓ Running & packaging JAR

REST API Development

✓ @RestController & @RequestMapping	✓ ResponseEntity for custom responses
-------------------------------------	---------------------------------------

✓ HTTP Methods: GET, POST, PUT, DELETE, PATCH	✓ HTTP Status codes (200, 201, 400, 404, 500)
✓ @GetMapping, @PostMapping etc.	✓ JSON serialization with Jackson
✓ @PathVariable & @RequestParam	✓ Global Exception Handling (@ControllerAdvice)
✓ @RequestBody & @ResponseBody	✓ Build tools — Maven goals (clean, install, package)

MINI PROJECT: Book Library REST API

Full CRUD REST API for books — Create, Read, Update, Delete. Custom exceptions (BookNotFoundException). Proper HTTP status codes. Test with Postman.

Tech: Spring Boot, REST API, Maven, Postman

Resources → YouTube:

■ Daily Code Buffer — Spring Boot

■ Java Brains Spring Boot

Resources → Docs:

■ spring.io/guides (official quick-start guides)

PHASE
3

Database Integration — SQL, JDBC, JPA & Hibernate

Weeks 4-5 | Days 31-42 | Intermediate

Weeks 4-5 | Days 31-42

SQL Fundamentals

✓ DDL — CREATE, ALTER, DROP, TRUNCATE	✓ Indexes — when & why to use
✓ DML — INSERT, UPDATE, DELETE, SELECT	✓ Transactions — ACID properties
✓ WHERE, ORDER BY, GROUP BY, HAVING	✓ Constraints — PK, FK, UNIQUE, NOT NULL
✓ JOINS — INNER, LEFT, RIGHT, FULL OUTER	✓ MySQL / PostgreSQL setup locally
✓ Subqueries & nested SELECT	

JDBC (Java Database Connectivity)

✓ JDBC architecture & drivers	✓ CRUD with JDBC
✓ DriverManager & Connection	✓ Transaction handling with JDBC
✓ PreparedStatement vs Statement	✓ Connection pooling concept
✓ ResultSet iteration	

Hibernate & JPA

✓ @Entity, @Table, @Column annotations	✓ JPQL vs native SQL queries
✓ @Id & @GeneratedValue strategies	✓ HQL basics
✓ Entity relationships: @OneToOne, @OneToMany, @ManyToMany	✓ EntityManager & persistence context
✓ Cascade types & FetchType (LAZY vs EAGER)	✓ Hibernate SessionFactory vs JPA

Spring Data JPA

✓ JpaRepository & CrudRepository	✓ Sorting with Sort
✓ Derived query methods (findByName, findByAgeGreaterThan)	✓ @Transactional annotation
✓ @Query annotation (JPQL & native)	✓ Service layer pattern

✓ Pagination with Pageable & Page

✓ spring.datasource.* configuration

MINI PROJECT: Employee Management System with DB

Extend previous API with MySQL database. Implement Department-Employee OneToMany relationship. Pagination & sorting on employee list. Filter by department.

Tech: Spring Boot, Spring Data JPA, MySQL, Hibernate

Resources → YouTube:

- Amigoscode — Spring Data JPA (English)
- Telusko JDBC Playlist

Resources → Practice:

- SQLZoo.net — all tracks
- LeetCode SQL Easy (10 problems)

MAJOR PROJECT: FinTrack — Personal Finance Tracker API

After Phase 3, you know enough to build a real project. FinTrack is a full CRUD REST API for managing personal income & expenses — perfect for fintech company interviews.

- User registration & login (basic, without JWT yet)
- CRUD for transactions (income/expense categories)
- Monthly summary reports (Streams + JPQL aggregation)
- Pagination, sorting, filtering by date range
- H2 for dev, MySQL for production profile
- Full Postman collection + Swagger docs
- Git with feature branches + proper commit history

Stack: Spring Boot, Spring Data JPA, MySQL, Swagger, Postman, Maven, Git

PHASE

4

Development Tools & Best Practices

Weeks 5-6 | Days 43-52 | Intermediate

Weeks 5-6 | Days 43-52

Git & GitHub

✓ git init, add, commit, push, pull

✓ .gitignore best practices

✓ Branching — git branch, checkout, merge

✓ Git rebase vs merge

✓ Merge conflicts & resolution

✓ GitHub Actions intro (CI basics)

✓ Pull Requests & code review flow

API Testing & Design

✓ Postman — Collections, Environments

✓ springdoc-openapi dependency

✓ Request chaining & variables in Postman

✓ API versioning strategies (v1, v2)

✓ Swagger/OpenAPI 3.0 documentation (@OpenAPI)

✓ REST best practices & naming conventions

IntelliJ IDEA Productivity

- ✓ Live templates & shortcuts
- ✓ Database plugin for SQL
- ✓ Debugger — breakpoints, step over/into
- ✓ Run configurations

Java Advanced Concepts

- ✓ Multithreading — Thread, Runnable
- ✓ String Pool & StringBuilder internals
- ✓ ExecutorService & ThreadPool
- ✓ Inner classes & Anonymous classes
- ✓ synchronized keyword & race conditions
- ✓ Enum with methods
- ✓ volatile & atomic variables
- ✓ Design Patterns — Singleton, Factory, Builder, Strategy

MINI PROJECT: Documented TODO App API

Build a full TODO/Task manager API with full Swagger documentation, versioned endpoints (v1), Git branching workflow, and Postman collection export.

Tech: Spring Boot, Swagger, Git, Postman

Resources → YouTube:

- TechWorld with Nana — Git (short crash course)
- Refactoring Guru — Design Patterns

Resources → Read:

- 12factor.net — methodology for modern apps

PHASE
5

Low-Level Design (LLD) & Object Design

Weeks 6-7 | Days 53-62 | Intermediate

Weeks 6-7 | Days 53-62

SOLID Principles

- ✓ S — Single Responsibility Principle
- ✓ I — Interface Segregation Principle
- ✓ O — Open/Closed Principle
- ✓ D — Dependency Inversion Principle
- ✓ L — Liskov Substitution Principle
- ✓ Practical refactoring with SOLID

Design Patterns in Java

- ✓ Creational: Singleton, Factory, Abstract Factory, Builder
- ✓ When to use which pattern
- ✓ Structural: Adapter, Decorator, Facade, Proxy
- ✓ Anti-patterns to avoid
- ✓ Behavioral: Strategy, Observer, Template, Command

LLD — Class & System Design

- ✓ UML Class Diagrams
- ✓ Design an ATM machine
- ✓ Identifying entities, attributes, relationships
- ✓ Encapsulation & abstraction in design
- ✓ Design a Parking Lot (classic problem)
- ✓ Composition vs Inheritance decision
- ✓ Design a Library Management System

Code Quality

- ✓ Clean Code principles (Robert Martin)
- ✓ Refactoring techniques
- ✓ Naming conventions in Java
- ✓ SonarLint static analysis setup
- ✓ Method size & Single Responsibility

MINI PROJECT: Library Management LLD

Design and implement a Library system with proper class hierarchy, design patterns (Factory for books, Observer for notifications, Strategy for late fees). Include UML diagram.

Tech: Core Java, Design Patterns, OOP

Resources → YouTube:

- Concept && Coding — LLD Playlist (Shrayansh)
- Refactoring Guru patterns

Resources → Books:

- Clean Code — Robert C. Martin (key chapters)

PHASE

6

Advanced Spring Boot — Security, DTOs & Production

Weeks 8-10 | Days 63-80

Weeks 8-10 | Days 63-80 | Intermediate-Advanced

DTO Pattern & Validation

- | | |
|--|---|
| ✓ DTO (Data Transfer Object) pattern — why & how | ✓ @NotNull, @Size, @Email, @Pattern |
| ✓ ModelMapper / MapStruct for DTO mapping | ✓ @ControllerAdvice for validation error response |
| ✓ @Valid & Bean Validation (JSR-380) | ✓ Custom validators |

Spring Security

- | | |
|--|--|
| ✓ Security filter chain concept | ✓ Role-based authorization (@PreAuthorize) |
| ✓ UserDetails & UserDetailsService | ✓ Method-level security |
| ✓ BCrypt password encoding | ✓ CSRF — when to disable & why |
| ✓ In-memory vs DB-based authentication | ✓ CORS configuration |

JWT Authentication

- | | |
|--|--|
| ✓ What is JWT — header, payload, signature | ✓ JWT validation filter (OncePerRequestFilter) |
| ✓ jjwt library (io.jsonwebtoken) | ✓ Refresh token strategy |
| ✓ JWT generation on login | ✓ Securing endpoints with JWT |

OAuth 2.0 & Social Login

- | | |
|---------------------------------------|-----------------------------|
| ✓ OAuth 2.0 flow (Authorization Code) | ✓ Login with GitHub |
| ✓ Spring Security OAuth2 Client | ✓ Handling OAuth2 user info |
| ✓ Login with Google | |

Advanced API Features

- | | |
|------------------------------------|--|
| ✓ Logging with SLF4J + Logback | ✓ Environment-specific configs |
| ✓ Centralized exception monitoring | ✓ Actuator — health, metrics, info endpoints |
| ✓ Profiles (dev, staging, prod) | ✓ Rate limiting concept (Bucket4j) |
| ✓ @Profile annotation | |

MINI PROJECT: Secure Auth Service

Full authentication system: Register/Login with JWT, BCrypt passwords, role-based access (USER/ADMIN), Google OAuth2 login, refresh tokens, protected endpoints.

Tech: Spring Security, JWT, OAuth2, Spring Boot

Resources → YouTube:

- Amigoscode — Spring Security + JWT (English)
- Daily Code Buffer — OAuth2

Resources → Docs:

- Spring Security Official Reference Docs

MAJOR PROJECT: JobBoard — Full-Stack Job Portal Backend (Resume Project)

This is your PRIMARY resume project for Phase 1 of job applications. A production-quality job board backend targeting fintech/startup hiring needs.

- Multi-role auth: CANDIDATE / RECRUITER / ADMIN with JWT + BCrypt
- Job posting CRUD (Recruiter), Job application flow (Candidate)
- File upload — resume PDF stored to AWS S3
- Search & filter jobs (location, salary, tech stack)
- Email notifications (JavaMailSender / SendGrid)
- Redis caching for popular job listings
- Rate limiting on public APIs
- Comprehensive Swagger docs, Postman collection
- Dockerized + GitHub Actions CI/CD + deployed to AWS EC2/RDS

Stack: Spring Boot, Spring Security, JWT, MySQL, Redis, AWS S3, Docker, GitHub Actions, AWS EC2

PHASE
7

High-Level Design (HLD) & System Design

Weeks 10-11 | Days 81-90 | Intermediate-Advanced

Weeks 10-11 | Days 81-90

Scalability Fundamentals

✓ Horizontal vs Vertical scaling	✓ CAP Theorem
✓ Load Balancers (Nginx, AWS ALB)	✓ BASE vs ACID
✓ CDN — Content Delivery Networks	✓ Eventual vs Strong consistency
✓ Caching strategies (L1, L2, distributed)	

Databases at Scale

✓ SQL vs NoSQL — when to choose	✓ Replication — master-slave
✓ Redis — caching, pub/sub, data structures	✓ Connection pooling (HikariCP)
✓ MongoDB basics	✓ Read replicas pattern
✓ Database sharding & partitioning	

Common System Design Problems

- | | |
|--------------------------------------|---------------------------------|
| ✓ Design URL Shortener (like Bit.ly) | ✓ Design News Feed |
| ✓ Design Notification System | ✓ Design a Payment System |
| ✓ Design Rate Limiter | ✓ Design WhatsApp / Chat System |

Architecture Patterns

- | | |
|---|---------------------------|
| ✓ Monolith vs Microservices trade-offs | ✓ Circuit Breaker pattern |
| ✓ Event-Driven Architecture | ✓ API Gateway pattern |
| ✓ CQRS Pattern | ✓ Service Mesh concept |
| ✓ Saga Pattern (distributed transactions) | |

MINI PROJECT: System Design Document — URL Shortener

Create a complete HLD document for a URL shortener: DB schema, caching layer (Redis), load balancing, API design, scaling strategy (10M users). Implement basic version.

Tech: Spring Boot, Redis, H2/MySQL, System Design

Resources → YouTube:

- Gaurav Sen — System Design Playlist
- Ashish Pratap Singh — HLD Concepts

Resources → Read:

- System Design Primer (GitHub)
- ByteByteGo newsletter

PHASE
8

DSA for Backend Interviews

Parallel Track | Weeks 2-14 | Ongoing Practice

Parallel Track | Weeks 2-14

Data Structures (in Java)

- | | |
|--------------------------------|--|
| ✓ Arrays & 2D Arrays | ✓ Binary Tree & BST |
| ✓ Strings & StringBuilder | ✓ HashMap internals (collision, load factor) |
| ✓ Linked List (Singly, Doubly) | ✓ Heap / PriorityQueue |
| ✓ Stack & Queue (ArrayDeque) | ✓ Trie (for autocomplete problems) |

Algorithms

- | | |
|------------------------------------|---|
| ✓ Binary Search & variants | ✓ Recursion & Backtracking |
| ✓ Sorting — Merge, Quick, Counting | ✓ Dynamic Programming (Memoization first) |
| ✓ Sliding Window pattern | ✓ BFS & DFS on graphs/trees |
| ✓ Two Pointer technique | ✓ Greedy algorithms |

Interview Focus Topics

- | | |
|--|--|
| ✓ LeetCode Top 150 Interview Questions | ✓ Java-specific: ConcurrentHashMap, volatile |
| ✓ SQL queries for interview | ✓ Time & Space complexity analysis (Big-O) |
| ✓ OOP design questions | |

MINI PROJECT: DSA Progress Tracker

Track your LeetCode progress. Aim: Easy x30, Medium x50, Hard x10 before interviews. Note: use C++ for practice (as your existing skill) or Java — both fine.

Tech: Java / C++, LeetCode

Resources → YouTube:

■ Striver — A2Z DSA Sheet (NeetCode equivalent for India)

Resources → Platform:

■ LeetCode.com

■ Codeforces (optional)

■ GFG Java DSA

PHASE

9

Docker & DevOps Basics

Weeks 11-12 | Days 88-96 | Intermediate

Weeks 11-12 | Days 88-96

Docker Essentials

- | | |
|--|---------------------------------------|
| ✓ What is Docker & why containers | ✓ Docker volumes & networks |
| ✓ Docker vs VMs | ✓ Dockerize Spring Boot app |
| ✓ Dockerfile — FROM, RUN, COPY, EXPOSE, CMD | ✓ Docker Hub — push & pull images |
| ✓ Build & run Docker images | ✓ Environment variables in containers |
| ✓ docker-compose.yml — multi-container setup | |

CI/CD Basics

- | | |
|--|---------------------------------|
| ✓ What is CI/CD & why | ✓ Automated testing in CI |
| ✓ GitHub Actions — workflow YAML | ✓ Jenkins overview (optional) |
| ✓ Build pipeline: test + build + Docker push | ✓ CD — deploy to cloud on merge |

Monitoring & Observability

- | | |
|---|------------------------------------|
| ✓ Spring Boot Actuator endpoints | ✓ Health checks & readiness probes |
| ✓ Prometheus + Grafana basics | ✓ Application metrics collection |
| ✓ Centralized logging (ELK stack concept) | |

MINI PROJECT: Dockerized App with CI/CD

Containerize your Auth Service from Phase 6. Write docker-compose with App + MySQL. Create GitHub Actions pipeline: on push to main — build, test, Docker push to Docker Hub.

Tech: Docker, docker-compose, GitHub Actions

Resources → YouTube:

■ TechWorld with Nana — Docker (FREE, highly recommended)

■ GitHub Actions Crash Course

Resources → Docs:

■ docs.docker.com — Get Started guide

AWS Core Services (Backend Dev Focus)

- | | |
|---|--|
| ✓ IAM — users, roles, policies | ✓ Elastic Beanstalk — easiest Spring Boot deploy |
| ✓ EC2 — launch instance, SSH, security groups | ✓ Route 53 — DNS basics |
| ✓ RDS — managed MySQL/PostgreSQL | ✓ VPC — subnets, internet gateway concept |
| ✓ S3 — object storage (file uploads from API) | ✓ CloudWatch — logs & alarms |

Deployment Workflow

- | | |
|--|---|
| ✓ Deploy Spring Boot JAR to EC2 | ✓ Environment variables via AWS Parameter Store |
| ✓ Connect app to RDS (not local MySQL) | ✓ Auto-scaling groups concept |
| ✓ Nginx as reverse proxy on EC2 | ✓ ECS (Elastic Container Service) — Docker on AWS |
| ✓ HTTPS with SSL (Let's Encrypt / ACM) | |

AWS Developer Tools

- | | |
|---|--------------------------------------|
| ✓ AWS CLI setup & basic commands | ✓ Lambda basics (serverless concept) |
| ✓ CodePipeline + CodeBuild (AWS native CI/CD) | ✓ API Gateway + Lambda |
| ✓ Secrets Manager for DB passwords | |

MINI PROJECT: Deploy Your App to AWS

Deploy your Dockerized Spring Boot app to AWS EC2 + RDS. Set up Nginx reverse proxy. Add SSL. Hook up S3 for profile picture upload. Live URL on your resume!

Tech: AWS EC2, RDS, S3, Docker, Nginx

Resources → YouTube:

- TechWorld with Nana — AWS for DevOps
- freeCodeCamp AWS Course (YouTube)

Resources → Platform:

- AWS Free Tier — use it! (1 year free)
- AWS Skill Builder (free training)

Microservices Fundamentals

- | | |
|---|--|
| ✓ Monolith to Microservices decomposition | ✓ API Gateway (Spring Cloud Gateway) |
| ✓ Domain-Driven Design (DDD) basics | ✓ Service Registry (Eureka / Consul) |
| ✓ Bounded Context concept | ✓ Client-side load balancing (Spring Cloud LoadBalancer) |
| ✓ Service per DB pattern | ✓ Config Server (centralized configuration) |

Inter-Service Communication

- | | |
|------------------------------------|-------------------------------|
| ✓ Synchronous: REST with OpenFeign | ✓ Kafka producers & consumers |
|------------------------------------|-------------------------------|

- | | |
|-------------------------------------|--|
| ✓ Circuit Breaker with Resilience4j | ✓ Kafka topics, partitions & consumer groups |
| ✓ Retry & Fallback patterns | ✓ RabbitMQ alternative (brief overview) |
| ✓ Asynchronous: Apache Kafka | |

Distributed Systems Patterns

- | | |
|---|------------------------------------|
| ✓ Distributed tracing (Zipkin + Micrometer) | ✓ CQRS with event sourcing |
| ✓ Saga pattern (choreography & orchestration) | ✓ Idempotency in distributed APIs |
| ✓ Outbox pattern for reliability | ✓ Centralized logging (ELK / Loki) |

Security in Microservices

- | | |
|--|------------------------------|
| ✓ JWT propagation across services | ✓ mTLS (mutual TLS) concept |
| ✓ OAuth2 with Keycloak (identity server) | ✓ API Gateway authentication |

Kubernetes (K8s) Basics

- | | |
|-------------------------------|--|
| ✓ Pods, Deployments, Services | ✓ Horizontal Pod Autoscaler |
| ✓ ConfigMaps & Secrets | ✓ Helm charts (package manager for K8s) |
| ✓ Ingress controllers | ✓ Deploy microservices to K8s (Minikube locally) |

MINI PROJECT: E-Commerce Microservices Platform

Build 4 services: User-Service, Product-Service, Order-Service, Payment-Service. API Gateway routing. Kafka for order events. Eureka service discovery. Docker + K8s deployment.

Tech: Spring Boot, Spring Cloud, Kafka, Docker, Kubernetes, Eureka

Resources → YouTube:

- TechWorld with Nana — Microservices
- Amigoscode — Spring Cloud

Resources → Read:

- Microservices.io patterns catalog

MAJOR PROJECT: PayFlow — Fintech Payment Microservices Platform (Flagship Project)

Your ultimate showcase project for Senior/Mid-Level applications. Demonstrates microservices, Kafka, cloud deployment — exactly what Razorpay/CRED/PhonePe look for.

- 5 Microservices: User, Wallet, Transaction, Notification, Gateway
- Kafka for async payment event processing (debit/credit flow)
- Saga pattern for distributed transactions (payment rollback on failure)
- Keycloak for centralized auth (OAuth2 + JWT across services)
- Distributed tracing with Zipkin
- Kubernetes deployment (Helm charts) on AWS EKS
- Prometheus + Grafana monitoring dashboard
- AI feature: Smart fraud detection alert (Spring AI)

- Full system design document (HLD + LLD)
- CI/CD pipeline with GitHub Actions

Stack: [Spring Boot](#), [Spring Cloud](#), [Kafka](#), [Keycloak](#), [Redis](#), [Docker](#), [Kubernetes](#), [AWS EKS](#), [Spring AI](#), [Prometheus](#), [Grafana](#)

PHASE
12

AI Integration with Spring Boot

Week 16+ | Days 115-125 | Advanced

Week 16+ | Days 115-125

Spring AI Framework

- | | |
|----------------------------------|---|
| ✓ Spring AI dependency & setup | ✓ Structured output parsing |
| ✓ OpenAI API integration | ✓ Streaming responses |
| ✓ ChatClient — prompt & response | ✓ Vector databases (Pinecone, pgvector) |
| ✓ Prompt templates in Spring AI | ✓ RAG — Retrieval Augmented Generation |

AI-Powered Features

- | | |
|--|---|
| ✓ AI chatbot endpoint in REST API | ✓ Semantic search with embeddings |
| ✓ Document Q&A; (upload PDF → ask questions) | ✓ Content moderation API |
| ✓ Code generation assistant | ✓ Image generation (DALL-E integration) |

MINI PROJECT: AI-Powered Backend Service

Add AI features to your project: smart search, chatbot endpoint, auto-summarize documents. Use Spring AI + OpenAI. This makes your resume stand out significantly!

Tech: [Spring AI](#), [OpenAI API](#), [Spring Boot](#), [Vector DB](#)

Resources → YouTube:

- Dan Vega — Spring AI Tutorials (official Spring team)

Resources → Docs:

- docs.spring.io/spring-ai (official docs)

Complete Roadmap Timeline — 2025 to 2027

Phase 0-1	Core Java	May 2025
Phase 2-3	Spring Boot + DB	Jul 2025
Phase 4-5	Tools + LLD	Sep 2025
Major Project 1	FinTrack API	Oct 2025
Phase 6-7	Security + HLD	Dec 2025
Major Project 2	JobBoard Platform	Feb 2026
Phase 8-9	DSA + DevOps	Apr 2026
Phase 10	AWS Cloud	May 2026
Phase 11-12	Microservices + AI	Aug 2026
Major Project 3	PayFlow Platform	Oct 2026
Interview Prep	DSA + Mock Interviews	Nov 2026
Target: First Job	8-12 LPA Fintech Role	Jan-Mar 2027

Final Success Tips

- 01 80/20 Rule: 80% building projects, 20% theory. Stop over-reading, start coding.
- 02 Networking: LinkedIn posts on your projects = inbound recruiter messages from fintech cos.
- 03 Parallel DSA: Do 3-5 LeetCode problems per week throughout all phases. Don't batch it.
- 04 Apply Early: Start applying after Phase 6 (Major Project 2 done). Don't wait for 'perfect'.
- 05 Resume Rule: Each project bullet = WHAT you built + HOW (tech) + IMPACT (users/scale).
- 06 Target Companies: Razorpay, CRED, PhonePe, Groww, Zerodha, BrowserStack, Atlassian India.
- 07 Platforms: Naukri.com (volume), Cutshort.io (startups), LinkedIn (referrals), AngelList.
- 08 Interview Prep: Revise all phases in last 2 months. Mock interviews on Pramp / interviewing.io.